UL2ME007 - Algorithms and Scientific Programming

Lab 3: Arrays and Masks

This lab is based on Course 2 and Tutorial 2 of the same name.

The necessary syntax is recalled in the Python course sheets

4 - Numpy arrays - vectorization

5 - Masks

From this practical, you only use the Spyder development environment (IDE). Don't forget to use all Spyder tools,

in particular

the variable explorer to view variables in memory,

the Ipython console to perform simple verification operations.

1 - Manipulation of a NumPy vector: temperatures during the month

2 - Matrix creation and manipulation

3 - Initialization of the Vandermonde matrix

1 - Manipulation of a NumPy vector: temperatures during the month

> Note: This exercise largely repeats Exercise 1 from Tutorial 2. We add plots to visualize the operations. To write the code that creates these plots, you can use the very similar examples from sheet 5 - Masks.

Here is a NumPy array containing the maximum temperature for each day of June 2026 in Paris, in °C:

```
import  numpy  as  np
import  matplotlib.pyplot    as  plt

Tjuin2026   = np.array([26.4    ,21.7  ,20.2  ,22.0  ,20. ,21.6  ,23.0  ,23.3  ,20.5  ,19.0  ,
19.9,\
```

26.2 ,28.5 ,26.8 ,28.6 ,29.0 ,32.4 ,36.9 ,36.3 ,35.8 ,36.9,\

37.7 ,37.4 ,40.6 ,39.8 ,37.3 ,37.2 ,31.3 ,28.8 ,29.6])

0. Plot temperatures according to the day of the month (from 1 to 30) in a dedicated graph. Don't forget the presentation (name of the axes...).

1. Show temperatures for the first 7 days and the last 7 days.

2. Using NumPy commands, display the temperature: (a) average, (b) minimum, (c) maximum, (d) average over the last 15 days.

3. Construct a mask indicating the days when the temperature exceeds 25°C. Display the temperatures of the corresponding days then their number.

4. Identify the days when the temperature was at least 5°C higher than the average for the month. Show these days and the temperatures of these so-called days

exceptionally hot.

5. Resume the initial graph in a new figure. Indicate on this graph the average with a horizontal line, and the exceptional days with

red round markers. Don't forget the legend.

```
In  [ ]: # YOUR CODE HERE
raise  NotImplementedError()
```

```
In  [ ]: # YOUR CODE HERE
raise  NotImplementedError()
```

```
In  [ ]: # YOUR CODE HERE
raise  NotImplementedError()
```

```
In  [ ]: # YOUR CODE HERE
```

```
raise  NotImplementedError()
```

2 - Matrix Manipulation [excerpt from the 2025 lab exam]

> Note: We use the following notation: the entries of an n x n matrix M are denoted M_ij, with 1 <= i, j <= n. Warning: this is the mathematical convention where the indices start at 1, whereas in Python they start at 0. You must therefore adapt your code.

1. Ask the user to enter an integer n > 2. The entry will be re-proposed as long as this condition is not verified. In the following we will enter n = 5.

2. Create an n x n matrix A such that A_ij = n - |i - j|. Display A.

3. Let alpha = 1/(2n + 2). Create an n x n matrix B such that

```
B_ij = I_ij + {
    -n*alpha,  if i = j = 1 or i = j = n
    -1/2,      if |i - j| = 1
     alpha,    if |i - j| = n - 1
     0,        otherwise
}
```

where I is the identity matrix of order n. Display B.

4. Calculate the matrix P, matrix product of A by B. Display P.

5. Using a mask, "clean" the matrix P by setting to zero any element with an absolute value less than 10-8. Display the new matrix P.

```
In  [ ]: # YOUR CODE HERE
raise  NotImplementedError()
```

3 - Initializations of the Vandermonde matrix

> Note: This exercise largely repeats Exercise 2 from Tutorial 2. We add timing tests to compare the different alternatives.

Let X = (x_1, x_2, ..., x_n) be a vector in R^n, with n >= 2. The Vandermonde matrix associated with X is the n x n matrix V[X] defined by

```
V_ij[X] = x_i^(j-1), for i = 1, ..., n and j = 1, ..., n.
```

In expanded form:

```
V[X] = [
  1   x_1   x_1^2   ...   x_1^(n-1)
  1   x_2   x_2^2   ...   x_2^(n-1)
  ...
  1   x_n   x_n^2   ...   x_n^(n-1)
]
```

1. We start a Python script.

```
import  numpy  as  np
X  = np.array([1   ,2 ,3,  4])  # example of initializing X
n  = np.size(X)
```

Complete it in such a way that it initializes the Vandermonde matrix V component by component. The code should work for any array of n real elements.

> Note: Python array indices start at 0, whereas the mathematical convention above starts at 1.

2. We now want to gain efficiency and initialize the matrix by filling it column by column. Write the code that carries out this implementation.

3. Using x_i^j = x_i^(j-1) * x_i, propose a variation of the previous initialization that avoids exponentiation and uses only multiplication.

4. Now propose a code which carries out the initialization line by line using the power vector p = np.arange(n) .

```
In  [ ]: # YOUR CODE HERE

raise  NotImplementedError()
```

We now want to compare these ways of doing things, and for this we will use the timeit library which allows us to measure the execution time of a block of

code:

```
import  timeit
nit  = 50  # number of block repetitions

def  bloc():
```

#... your code here... pass

```
temps  =  timeit.timeit(bloc,number=nit)      # execution time averaged over nit repetitions
```

print("Execution time:", temps, "seconds")

> Note: we see that we use the syntax of functions with the keyword def to define a block of code which will be called next. The chapter to come on the functions will return in more detail to the associated syntax.

6. Complete this code to measure the execution time of initializations 1 and 2 above.

```
In  [ ]: # YOUR CODE HERE

raise  NotImplementedError()
```

To initialize a small 4×4 matrix like above, all methods are very fast. To properly compare these methods, we will measure this time

```
for n × n matrices with increasingly large n (always Vandermonde matrices with X=np.arange(1,
n+1) for simplicity). We choose the 4
```

following values: n = 5,10,20,40. We therefore have 4 methods to compare for 4 values of parameter n. We will therefore store the time values in a

4 × 4 time table (each line corresponds to a method) to then display the results in graph form.

7. Using a loop over n, calculate the 16 execution time values and populate the time array.

8. Display the results in a graph with n on the abscissa, execution times on the ordinate, and a curve for each method. What do you notice?

How do you explain it?

> Note: as n varies exponentially (in powers of 2), it is better to use a logarithmic scale plot with plt.loglog rather than plt.plot .

```
In  [ ]: # YOUR CODE HERE
raise  NotImplementedError()

In  [ ]: # YOUR CODE HERE

raise  NotImplementedError()

In  [ ]: # YOUR CODE HERE
raise  NotImplementedError()
```